Ders 5 Genişletilmiş Notu: Nesneye Yönelik Çözümleme ve Tasarım: Use Case, Sınıf, Sıralama ve Etkinlik Diyagramları

Ders 5 Genişletilmiş Notu: Nesneye Yönelik Çözümleme ve Tasarım: Use Case, Sınıf, Sıralama ve Etkinlik Diyagramları

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Dersin Yeri ve Önceki Konuların Hatırlatılması
- 2. Gereksinim Mühendisliğinin Yedi Adımı
- 3. Nesneye Yönelik Çözümleme ve Alan Analizi
- 4. Kullanım Öyküsü ve Kullanım Senaryosu Belgesi
- 5. Use Case Diyagramı
- 6. Sınıfların ve Etkileşimlerin Belirlenmesi
- 7. Kavramsal Sınıf Diyagramı (Alan Modeli)
- 8. Analiz Aşamasının Belgeleri
- 9. Sıralama (Sequence) Diyagramına Giriş ve CASE Araçları
- 10. Etkinlik (Activity) Diyagramı
- 11. Tasarım Aşaması: Kahve Sitesi Sıralama Diyagramı
- 12. Tasarım Sınıf Diyagramı
- 13. Tasarımdan Kodlamaya
- 14. Sıralama Diyagramının Yapısı: Basit Örnek
- 15. Sınıflar Arası İlişkiler
- 16. Kahve Sitesi Use Case Örneği
- Örnek Sorular

1. Dersin Yeri ve Önceki Konuların Hatırlatılması

Hoca derse, bir önceki haftada gereksinim analizinin (requirements analysis) konuşulduğunu ve bu analizin bütün adımlarının nesneye yönelik (object-oriented) yaklaşımla gerçekleştirileceğini hatırlatarak başladı. Plan şuydu: Önce 5-10 dakika geçen haftanın konuları hızlıca tekrar edilecek, sonra nesneye yönelik yaklaşımla modellemenin nasıl yapıldığına geçilecekti.

Geçen hafta use case konusunda iki orijinal slayt seti kullanılmıştı: biri use case oluşturma, öbürü alan modeli (domain model) üzerineydi. Hoca bu setlerin Larman tarafından hazırlanmış orijinal slaytlar olduğunu, use case'lerin genelden ayrıntıya doğru nasıl yazıldığını anlattığını söyledi. Bu derste aynı konuların Türkçeleştirilmiş slaytları üzerinden gidildi.

Hoca, yazılım süreç modellerinin ilk derslerde kapsamlı işlendiğini, bu yüzden üzerinde durmayacağını söyledi. O derslerde çevik (agile) ilkeler, çevik yöntemlerden aşırı programlama (extreme programming, XP) ve Scrum'ın özellikleri görülmüştü. Geçen hafta ise gereksinim mühendisliği (requirements engineering) işlenmişti.

2. Gereksinim Mühendisliğinin Yedi Adımı

2.1 Gereksinim Analizi ve SRS Belgesi

Hocanın tanımıyla gereksinim analizi, üzerinde çalışılmaya başlanacak projenin amaçlarını, boyutlarını ve etkilerini belirleme çalışmasıdır. Bunun öncesinde bir fizibilite (feasibility) çalışması da yapılır.

Gereksinim analizinin sonunda bir SRS belgesi oluşturulur. SRS, yazılım gereksinim belirtimi (Software Requirements Specification) demektir; ayrıntılı gereksinim bilgilerinin dökümüdür. SRS raporunun nasıl yazılacağı konusunda hoca iki kaynak verdi:

- **IEEE 830:** SRS'in standart yapısını gösterir.
- **2011 tarihli ortak standart:** ISO, IEC ve IEEE arasında 2011'de yapılan çalışmayla daha farklı ve daha kapsamlı bir standart oluşturulmuştur.

  Hocanın vurgusu: Proje raporlarında bu standartları, en azından özetleyerek de olsa, dikkate alan bir belge hazırlanması istenecek.

  Ek bilgi (derste anlatılmadı): Hocanın andığı 2011 tarihli ortak standart büyük olasılıkla ISO/IEC/IEEE 29148'dir. Bu standart, IEEE 830-1998'in yerini almıştır. Altyazıda "IEK" diye geçen kurum IEC'dir (International Electrotechnical Commission).

2.2 Pressman'a Göre Yedi Adım

Gereksinim mühendisliğinin adımları daha önce Pressman'ın kitabındaki başlıklarla görülmüştü. Toplam yedi adım vardır:

1. Başlangıç (inception).
2. Bilgi toplama (elicitation).
3. İşleme ya da modelleme (elaboration).
4. Pazarlık (negotiation).
5. Tanımlama (specification): SRS belgesinin oluşturulması.
6. Doğrulama (validation).
7. Gereksinim yönetimi (requirements management).

Son adım, doğrulamadan sonra gereksinimlerin yazılım geliştirme sürecinin diğer adımları boyunca nasıl takip edileceğini, izleneceğini ve yönetileceğini kapsar.

Hoca bunu sistem analizi dersindeki yaklaşımla karşılaştırdı. Orada gereksinim analizi daha basit, üç adım olarak görülmüştü: bilgi toplama, modelleme ve belge oluşturma. Burada daha fazla ayrıntıya girilir. Bu aşamada yazılım ekibi müşteriyle birlikte çalışmak zorundadır.

2.3 Başlangıç ve Bilgi Toplama

Başlangıç aşamasında "Bu işe neden giriyoruz, bu işe başlama gereksinimimiz nedir?" sorusu cevaplanır. Gerekçelendirme (justification) yapılır ve fizibilite sonuçları verilir.

Ardından sistem hakkında **bilgi toplamaya** başlanır. Hoca burada öğrencilere bilgi toplama yollarını sordu; cevaplar yüz yüze görüşme (interview), anket ve yerinde inceleme oldu (bkz. Örnek Sorular). Yerinde incelemede, daha fazla ayrıntı gerektiğinde, örneğin 3-4 gün boyunca sürecin içinde bulunularak işlerin gerçekte nasıl yürüdüğü izlenir.

Bilgi toplamada uyulacak ilkeler şunlardır:

- Gereksinimler hakkında ayrıntılı bilgi **tüm paydaşların** (stakeholders) katılımıyla elde edilir.
- Paydaşlar, sistemden doğrudan etkilenenlerdir: müşteriler, yönetim, sponsor, uç kullanıcılar ve yazılım geliştiriciler.
- Paydaşların hepsinin bakış açısı farklıdır. Hoca, bu bakış açılarının tanımlanmasına dayanan farklı yazılım geliştirme metodolojileri de olabileceğini ekledi.
- Toplantılara hazırlık ve katılım kuralları önceden belirlenmelidir. Yerinde izleme yapılacak ortamlar da belirlenmelidir.

2.4 İşleme (Modelleme)

İşleme (elaboration) aşamasında toplanan bilgiler işlenir ve etkileşimlerin nasıl olacağı modellenir. Gereksinim analizinde kullanılan araçlar şunlardır:

- Kullanım senaryoları (use case'ler).
- Kullanım senaryosu belgeleri.
- Kavramsal sınıf diyagramları.

Hoca, modelleme sırasında "normal gereksinim, beklenen gereksinim" gibi ayrımlar yapılsa da asıl işlevsel (functional) ve işlevsel olmayan (non-functional) gereksinimlerin SRS belgesinde ortaya konduğunu belirtti.

2.5 Pazarlık

Pazarlık (negotiation) aşamasında farklı bakış açılarındaki kişilerin farklı anlatımları değerlendirilir. Birbiriyle çelişen istekler varsa açıklığa kavuşturulur. Bu aşamada yazılım geliştiriciler de işin içindedir ve geliştiricilerle istekte bulunan müşteriler ve uç kullanıcılar arasında bir fikir birliğine varılır. Sorulan sorular şunlardır:

- İstenen her şey teknik olarak yapılabilir mi?
- Teknik olarak yapılabilse bile ekonomik açıdan maliyeti çok yüksek bir ürün mü ortaya çıkar?

Bu iki yön dengelenerek istenenin ne kadarının yapılabileceğine karar verilir.

2.6 Tanımlama ve Doğrulama

Tanımlama (specification) aşamasında gereksinimler yazıya dökülür. Bu belgeler şunları içerebilir: konuşma dilinde yazılmış metinler, kullanıcı senaryoları, şemalar, diyagramlar ve matematiksel gösterimler. Bir prototip oluşturulmuşsa o da eklenir. Bunların hepsi SRS belgesinde yer alabilir.

Doğrulama (validation) aşamasında şu sorular sorulur: Gereksinimler açıkça anlatılmış mı? Eksik ya da hata var mı? Gereksinimler gerçekçi mi? Ayrıca pazarlıkta geliştiriciler bazı isteklerin mümkün olmadığını ya da maliyetli olduğunu söyleyerek değişiklik istemişse, bu değişiklikler yapılmış mı? Belirtimde hâlâ yapılamayacak bir şey kalmış mı? Bunlar ortaya konur.

2.7 Gereksinim Yönetimi ve İzlenebilirlik Tablosu

Yönetim aşamasında, yazılım geliştirme süreci boyunca gereksinimlerde değişikliğin nasıl yapılacağı belirlenir. Yeni gereksinim eklemek, var olanı değiştirmek ya da önem sırasını değiştirmek gerekirse bunun hangi şartlarla yapılacağının önceden, resmi bir sözleşmeyle belirlenmiş olması istenir. Hoca bu değişiklik kurallarının doğrulama aşamasında belirlenmesi gerektiğini de söyledi.

Yönetimin bir parçası da izleme işidir. **İzlenebilirlik tablosu** (traceability matrix), gereksinimleri modellemede, tasarımda ve gerçekleştirmede (kodlamada) takip eder. Hangi gereksinimin hangi modülle, hangi paketle, hangi sınıfla ya da veri tabanındaki hangi tabloyla sağlandığını gösterir.

> **Hocanın vurgusu:** İzlenebilirlik tablosu projelerde de istenecek. Her gereksinimin hangi modülle, hangi sınıfla, hangi veri tabanı tablosuyla karşılandığı gösterilmeli.

Hoca bu tekrarın ardından use case diyagramlarına geçmeden önce öğrencilere use case bilgilerini sordu. Bir öğrenci daha önce SRS belgesi hazırlamadığını ve use case diyagramı kullanmadığını, UML'i biraz veri tabanı dersinden bildiğini söyledi. Hoca, use case'lerin geçen hafta ve ondan önceki hafta işlendiğini, Larman'ın orijinal slaytlarının da sayfaya yükleneceğini belirtti.

3. Nesneye Yönelik Çözümleme ve Alan Analizi

3.1 Çözümlemenin Amacı

Nesneye yönelik çözümleme (object-oriented analysis) süreci, sistemi anlamak içindir: Sistemde hangi parçalar ve hangi temel elemanlar vardır, bunlar anlaşılır. İlk iş uygulama alanının, yani problem alanının anlaşılmasıdır.

Hoca uygulama alanını örneklerle açıkladı. Öğrenci bilgi sistemi yapılacaksa, bu sistemde hangi işlerin yapıldığı ve hangi "sözlüğün" kullanıldığı, yani alandaki kavramların neler olduğu anlaşılmalıdır. Hastane bilgi sistemi yapılacaksa oradaki kavramlar ve müşteriyle anlaşmanın yolları öğrenilmelidir. Analist, kullanıcılarla konuşabilecek hale geldikten sonra onların gereksinimlerini anlar ve uygulamadaki sınıfları belirler.

3.2 Alan (Domain) Analizi

Alan analizi aşamasında, gereksinimlerden önce problem alanıyla ilgili araştırmalar yapılır. Öğrenci bilgi sistemi örneği üzerinden hoca şunları saydı:

- **Literatür:** Bu konuda hangi kavramlar var?
- **Benzer uygulamalar:** Başka hangi uygulamalar var, hangi üniversiteler hangi sistemleri kullanıyor?
- **Anketler:** Kendi üniversitemizin istekleri doğrultusunda anket yapıldı mı? Var olan sistemin hatalı ya da zor kullanılan tarafları neler? Yeni sistem hangi nedenlerle isteniyor?
- **Uzman görüşü:** Bu konuda uzman olan kişilerin tavsiyeleri.

Bu araştırmalardan sonra gereksinimler belirlenir.

3.3 NextGen POS Örneği: Müşterinin Doğal Dildeki İstekleri

Hoca, Larman'ın temel kitaplarındaki NextGen POS adlı perakende satış sistemi örneğini kullandı. Bu örnekte müşterinin istekleri doğal konuşma diliyle ortaya konur. Müşterinin anlatımı şöyledir:

- Eski yazılım ihtiyaçlarımızı karşılamadığı için donanımlarımızı da yenileyerek perakende satış programımızın yeniden yazılmasını istiyoruz.
- Program, kasada yapılan alışveriş işlemlerine yardımcı olmalı. Yapılan her işlem program tarafından saklanmalı.
- Mali bilgiler harici bütçe sistemine, mal çıkış bilgileri ise envanter sistemine iletilmeli. Yani sistem diğer sistemlerle konuşabilmeli.
- Saklanan her işlem üzerinde daha sonra raporlama ve analiz yapılabilmeli.
- Müşteriye bunun karşılığında bir fiş verilmeli.
- Yapılan her satış için KDV hesaplanıp ayrıca belirtilmeli.
- Birden fazla şube varsa şubelerin düzenlenmesi ve merkezi bir sunucunun kullanılması isteği de eklenebilir.

Bu paragraf, bilgi toplama aşamasındaki görüşmeler, toplantılar ve yerinde incelemeler sonunda elde edilen bilgilerdir. Bilgiler önce böyle doğal dilde yazılmış bir paragrafta tutulur. Gereksinimler bu paragraftan çıkarılır.

4. Kullanım Öyküsü ve Kullanım Senaryosu Belgesi

4.1 Kullanım Öyküsü Nedir?

Doğal dildeki ifadelerden sonra **kullanım öyküsü** (use case metni) oluşturulur. Kullanım öyküsü, programın yapacağı işi ayrıntılı ve kurallara bağlı olarak anlatmaktır. Amaç, az önceki serbest paragrafı daha kurallı, yazılım geliştiricilerin tasarım yaparken daha iyi anlayıp takip edebileceği bir hale getirmektir.

Kullanım öykülerinin amaçları şunlardır:

- Ürünün beklenen işlevlerini ve çalışma ortamını belirlemek.
- Son kullanıcı ile yazılım ekibi arasında bir anlaşma zemini oluşturmak. Hoca bunun pazarlık aşamasında gerçekleştiğini hatırlattı.

- Sistemle kullanıcıların birbirini nasıl etkileyeceğini tanımlamak.
- Kabul testlerini (acceptance tests) kolaylaştırmak. Testler sırasında kullanım öyküleri üzerinden gidilerek istenen gereksinimlerin yerine getirilip getirilmediği daha kolay sınanır.

4.2 Senaryo Belgesinin Bölümleri

Bir kullanım senaryosunda üç bölüm bulunmalıdır: giriş bölümü, ana senaryo ve alternatif senaryolar.

Giriş bölümünde sırasıyla şunlar yer alır: öykünün adı, birincil aktör (primary actor), ilgililer ve her birinin ilgi alanları, ön koşullar ve son koşullar. Ardından ana öykü, öğrencilerin önceki derslerden aşina olduğu biçimde, bir algoritma gibi baştan sona sıralı adımlar halinde verilir. Bir problem çıktığında izlenecek adımlar alternatif adımlardır.

Hocanın slayttaki satış işlemi örneği şöyledir:

Alan	İçerik
Senaryo adı	Satış işlemi
Birincil aktör	Kasiyer
İlgililer ve ilgi alanları	Kasiyer, satıcı, müşteri
Ön koşul	Kasiyer kimliğini doğrulamış olmalı
Son koşul	Ödeme tahsil edilmiş olmalı
Ana senaryo	1. Müşteri alacağı ürünlerle kasaya gelir. 2. Satış işlemi başlatılır. …
Alternatif akış	3a. Barkod geçersizse sistem uyarı mesajı verir ve kayıt girişini reddeder. Müşteri bir kalemi alışverişten çıkarmak isterse kasiyer o ürünün barkodunu okutur, sistem güncel toplamı gösterir.

Hoca iki önemli noktayı da belirtti:

- Kullanım öyküsünde alan modeli gözden kaçırılmamalıdır. Her şey alan modelinde verilenler kapsamında olmalıdır.
- Aktör bir insan olabileceği gibi yazılım ya da donanım gibi bir varlık da olabilir.

5. Use Case Diyagramı

5.1 Çizim Kuralları

Kullanım öyküleri grafik olarak da gösterilebilir. Bu gösterim use case diyagramıdır. Hoca çizim kurallarının geçen hafta işlendiğini hatırlatarak şunları saydı:

Eleman	Gösterim
İşlem (senaryo)	Elips. Her işlem ayrı bir elipsle gösterilir.
İnsan aktör (kullanıcı)	Çöp adam.
İnsan olmayan aktör (yazılım, donanım elemanı, algılayıcı, sensör)	Dikdörtgen.
Aktörle işlem arasındaki etkileşim	Çizgi.

Slaytta bir kasiyerin satış senaryosunu gerçekleştirdiği ve bu senaryonun ödemeyi de içine aldığı bir örnek vardı. Ayrıca provizyon işleminin ödemenin uzantısı olduğu gösteriliyordu. Bir donanımı temsil eden aktörün de use case C'yi kullandığı görülüyordu.

5.2 Senaryolar Arası İlişkiler: «extend» ve «include»

İşlemler arasında iki farklı ilişki vardır: «extend» ve «include». Ok yönü aynı zamanda ilişkinin okunma yönüdür.

- **«extend» (genişletme):** Use case B, use case A'nın genişletilmişidir. B işlevi, A işlevi yürütülürken oluşabilecek bir sapma, genişleme ya da uzanımdır. A ana akışı gösterir; ana akıştaki bir seçenek ya da alt akış gerekirse bu uzantı yapılır. Mutlaka gerçekleşmez.
- **«include» (içerme):** Use case C işlevi, A tarafından içerilir. A gerçekleştiriliyorsa C de ona dahildir.

Hoca ayrıca senaryolar arasında benzeşim (özelleştirme ya da genelleştirme) ilişkileri de olabileceğini söyledi.

Hoca use case çizimi ve belgesiyle ilgili soru olup olmadığını sordu, soru gelmeyince 10 dakika ara verildi.

6. Sınıfların ve Etkileşimlerin Belirlenmesi

Aradan sonra sınıfların belirlenmesine geçildi. Hoca, sınıf ve nesnenin ne olduğuna kadar geriye gitmeyeceğini, bunların temel bilgiler olduğunu söyledi. Sınıfları belirlerken sorulacak sorular şunlardır:

- Hangi sınıflar var?
- Hangi eylemler gerçekleştiriliyor?
- Sınıfların sorumlulukları nelerdir?

Sorumluluklar genelden özele doğru dağıtılır; yani kalıtım hiyerarşisine dikkat edilir. Bilgi ve o bilgiyle ilgili davranışlar aynı sınıfta yer almalıdır; bu kapsülleme (encapsulation) kavramıdır. Kısacası sınıflar, nesneye yönelik yaklaşımın temel özellikleri olan kapsülleme ve kalıtım gözetilerek oluşturulur.

Bu sırada etkileşimler de belirlenir. **Etkileşim**, bir nesnenin üzerine düşen sorumluluğu yerine getirmek için başka bir nesneye mesaj göndermesidir. Sınıf diyagramında nesneler arasındaki ilişkiler de gösterilmelidir: bağlantılar, özelleştirme, genelleştirme ve sınıf-nesne ilişkileri. Hoca, nesneye dayalı derslerden eksiği olanların bunları orijinal use case slaytlarından tamamlaması gerektiğini söyledi.

Hoca, geçen hafta sınıflar arasındaki ilişkilerde bir "tersi" kavramından da bahsedildiğini hatırlattı.

Sınıflar belirlenirken sınıf, sorumluluk ve işbirliği tanımları, yani CRC (Class-Responsibility-Collaborator) kartları akılda tutulmalıdır.

7. Kavramsal Sınıf Diyagramı (Alan Modeli)

Hoca POS örneğinin alan modelini anlattı. Altyazıdan çıkan ilişkiler şunlardır:

- Bir satış kalemi bir ürünü içerir. Satış kaleminde tek bir ürün tanımı olmalıdır.
- Stoktaki her ürün, ürünün tanımı, fiyatı, kodu ve barkoduyla tanımlıdır.
- Bir satışın içinde birden fazla satış kalemi olabilir. İlişki bire çok da olabilir, bire bir de. Yani kasadan geçen bir satışta tek bir kalem de olabilir, birden fazla kalem de.
- Bir satış bir ödeme içerir.
- Satış, kasa tarafından kaydedilir.
- Her şubedeki işlemler kasada gerçekleştirilir.

Bunu metin olarak şöyle özetleyebiliriz:

Sınıf	Öznitelikler	İlişki
Ürün (ürün tanımı)	tanım, fiyat, kod, barkod	SatışKalemi bir Ürün'ü içerir
SatışKalemi	miktar (BELİRSİZ)	Satış birden fazla SatışKalemi içerir (1..*)
Satış	tarih (BELİRSİZ)	Satış bir Ödeme içerir; Kasa, Satış'ı kaydeder
Ödeme	miktar (BELİRSİZ)	
Kasa		Şube içinde
Şube	ad, adres (BELİRSİZ)	

Hocanın bu diyagramla ilgili vurguladığı nokta şudur: Bu bir alan modeli, yani kavramsal sınıf diyagramıdır. Sınıf kutularının altında her sınıf tarafından gerçekleştirilen işlemler yoktur. Tam (tasarım) sınıf diyagramında ise alttaki üçüncü kutuda sınıfın gerçekleştireceği işlemler, yani fonksiyonları yazılır.

> **Hocanın vurgusu:** Kavramsal sınıf diyagramında yalnızca sınıfın adı, sınıftaki özellikler (öznitelikler) ve sınıflar arasındaki ilişkiler gösterilir. İşlemler yazılmaz.

8. Analiz Aşamasının Belgeleri

Böylece analiz aşaması tamamlanmış olur. Hoca, nesneye yönelik analiz sürecinin şu belgelerle belgelendirildiğini söyledi:

- UML kullanım şemaları (use case diyagramları).
- Kullanım senaryosu belgeleri.
- UML (kavramsal) sınıf şemaları.
- Veri tabanı tanımları olacaksa ER (varlık-ilişki) diyagramı.

> **Hocanın vurgusu:** Bunlar analiz aşamasında mutlaka olması gereken belgelerdir.

Hoca, tasarım aşamasında sayılan bazı etkinlik (activity) diyagramlarının analizde de yer alabileceğini, ama şimdilik bunların daha çok tasarım aşamasına ait sayıldığını belirtti.

9. Sıralama (Sequence) Diyagramına Giriş ve CASE Araçları

9.1 Sıralama Diyagramı

Sıralama diyagramlarının (sequence diagram) hazırlanmasına aslında analiz aşamasında başlanır, ama bunlar tasarım belgeleri arasında sayılır. Türkçede ardışıl diyagram, sıralama diyagramı ya da sıralama şeması adları kullanılır; hepsi aynı şeydir.

Yapısı şöyledir:

- Nesneler, işleme girecekleri sırayla ve hangi sınıftan oldukları belirtilerek en üste dizilir.
- Sınıfların işleme giriş sırası aşağı doğru gösterilir.
- Bir mesaj, bir metodun çağrılmasıdır. Geri dönüş noktalı (kesikli) çizgiyle gösterilir.
- Koşullu mesaj olabilir; bu bir `if` yapısıdır. Bir işin nasıl gerçekleştiğini göstermek için döngü de olabilir.

Hocanın özetiyle, biri “sıralama diyagramı çiz” dediğinde istenen şudur: Sınıfların birbirleriyle hangi metodu çağırarak ilişki kurduğunu, işleme nasıl ve hangi sırada girdiğini anlatan bir diyagram. Hoca bunun UML diyagramları içinde önemli bir diyagram olduğunu söyledi. Örneklerin nesneye yönelik derslerde verildiğini ve bu konularla ilgili bütün bir uygulamanın da öğrencilere yaptırılacağını ekledi.

9.2 CASE Araçları ve Violet

Hoca ilk derslerde anlatılan CASE (Computer Aided Software Engineering, bilgisayar destekli yazılım mühendisliği) yaklaşımını hatırlattı. CASE araçları, diyagramları birbiriyle ilişkilendirerek daha standart biçimde çizmeyi sağlayan ortamlardır.

Diyagramları çizmek için Violet önerilmişti. Hoca, Violet’ın “menekşe” sözcüğünün İngilizcesi olduğunu söyledi. Violet, UML diyagramlarının daha doğru, daha standart ve kurallara uygun çizilmesini sağlayan bir yazılımdır. Açık kaynaklı olduğu için önerilmiştir. Başka bir programa alışkın olanlar onu da kullanabilir.

> **Hocanın vurgusu:** Diyagramların standartlara uygun çizilmesi için böyle bir araç kullanılması isteniyor.

10. Etkinlik (Activity) Diyagramı

10.1 Ne İşe Yarar?

Etkinlik diyagramları (activity diagram, etkinlik şemaları) özellikle işlemlerin aynı anda iki (ya da daha fazla) ortamda nasıl yürüdüğünü göstermek için kullanılır. Yapı olarak akış diyagramına benzer, ama akışların farklı ortamlarda nasıl yürüdüğünü gösterir. Hoca, birçok kaynakta etkinlik diyagramlarının analiz aşamasında çizildiğinin gösterildiğini de belirtti.

10.2 Elemanlar

Hocanın saydığı elemanlar şunlardır:

- **Denetim akışı:** Etkinlikler arasındaki akış.
- **Etkinlik başlangıcı:** Başlangıç işareti.
- **Sinyal alma:** Bir olayın başlaması için sinyalin alınması. Akış, sinyal alana kadar bekler. Zamanla başlayacak olaylar bu şekilde gösterilir.
- **Eylem:** Herhangi bir işin gerçekleştirilmesi.
- **Karar:** `if` yapısı.
- **Eşgüdüm çizgisi:** Aynı anda birden fazla ortamda yürüyen işlerin eşgüdümünün başladığı ve bittiği yer.
- **Etkinlik sonu.**

Etkinlik şemaları bir başlangıç işaretiyle ya da sinyal alma işaretiyle başlar.

10.3 Eşgüdüm: Çatallanma ve Birleşim

Eşgüdüm çizgisinin iki kullanımı vardır:

- **Birleşim (join):** Farklı işlerin bitişini gösteren akışlar bir noktada birbirini bekler. Eşgüdüm çizgisine varan bir akış, çizgiyi geçmeden önce diğer akışların hepsini bekler. Sonra tek işlem olarak devam edilir.
- **Çatallanma (fork):** Tek bir işlemden gelen akış, eşgüdüm çizgisinden sonra farklı etkinliklere doğru ayrılabilir.

Yani eşgüdüm yapısı, hangi ortamda hangi etkinliğin hangi kurallarla gerçekleştiğini belirtmek ve farklı ortamlara akış sağlamak için kullanılır.

10.4 Örnek: Uçağa Biniş

Slayttaki örnek uçağa biniş işlemleriydi. Yer hizmetlerinde ve müşteri hizmetlerinde farklı yürüyen iş adımları vardır. Her ortam ayrı bir kulvarda gösterilir. Diyagram şu soruları cevaplar: Bu etkinliklerden hangisi hangi ortamda yürüyecek? Farklı ortamlarda olsalar bile aynı anda yürüyecek işlemler var mı?

Hoca durum şemalarına (state diagram) bu derste girilmeyeceğini, önümüzdeki hafta işleneceğini söyledi.

11. Tasarım Aşaması: Kahve Sitesi Sıralama Diyagramı

11.1 Tasarımda Neler Yapılır?

Hoca ekranı değiştirip, sistem analizi dersinde de anlatılan konuları hatırlatarak devam etti. Kaynak, hocanın Göksel Hoca ile yazdığı kitaptı. Senaryolar ve sözleşmeler analizden gelir. Tasarım aşamasında:

- Önce sorumluluklar belirlenir.
- Tasarım kalıpları (design patterns) kullanılacaksa bunlar belirlenir.
- Sorumlulukların atanmasıyla tasarım modellemesine geçilir.

Tasarımın modelleri ardışıl diyagramlar ve yazılım (tasarım) sınıf diyagramlarıdır. İşbirliği diyagramları (collaboration diagram) sıralama diyagramlarının eşdeğeridir. Hoca bu derste ardışıl diyagramların tercih edildiğini, çünkü işlemlerin sırasını da gösterdiklerini söyledi. Yazılım

sınıf diyagramları analizde gösterilmemişti; analizde kavramsal sınıf diyagramları vardı. Tasarımda sınıf diyagramı daha kapsamlı yer alır.

Tasarım kalıplarının önümüzdeki hafta görüleceği söylendi.

11.2 Sepete Kahve Ekleme

Hocanın gösterdiği ardışıl diyagram, bir kahve satış sitesinde alışveriş sırasında sepete bir ürün ekleme işlemiydi. Altyazıdan çıkan akış şudur:

- Diyagramdaki sınıflar: tasarım kalıbından gelen Denetçi sınıfı, KahveKatalog, Kahve, Sipariş ve sipariş kalemleri.
- "Kahve ekle" dendiğinde bu kahvenin tanımı alınır. Kahve katalogdan seçilir ve stoktaki bilgileriyle tanımlanır.
- Sipariş, Sipariş sınıfı içinde eklenir.
- Sipariş kalemleri yaratılır ve stoktaki tanımlarıyla oluşturulur.
- Diyagram üzerinde hangi adımda hangi tasarım kalıbının kullanıldığı notlarla gösterilmişti: yaratıcı, uzman ve denetçi tasarım kalıpları.

Kısa nottaki slayt okumasına göre mesaj sırası şöyledir:

No	Mesaj	Gönderen → Alan	Tasarım kalıbı
1	kahveEkle(id, miktar)	(kullanıcı arayüzü) → :Denetçi	Denetçi
2	tanim = tanimAl(id)	:Denetçi → :KahveKatalog	Uzman
3	tanim = bul(id)	:KahveKatalog → :Map<Kahve>	
4	siparişEkle(tanim, miktar)	:Denetçi → :Sipariş	
5	yarat(tanim, miktar)	:Sipariş → sk:SiparişKalemi	Yaratıcı
6	ekle(sk)	:Sipariş → siparişKalemleri:List<SiparişKalemi>	

11.3 Döngü ve Özel Kahve

Hoca, ürün ekleme işleminde bir döngü de kullanılabileceğini söyledi, çünkü işlem birden fazla sipariş için yapılacaktır. Özellikle özel kahve siparişinde, yani standart bir kahve değil de birden fazla kahvenin karıştırılarak satıldığı bir siparişte, içeriye birden fazla sipariş kaleminin eklenmesi gerekir. Bu, n tane kahvenin birleştirilerek satılacağı anlamına gelir. Bu yüzden sıralama diyagramında bu kısım bir döngü (loop) içinde gösterilir.

Hoca bu derste işbirliği diyagramının kullanılmayacağını tekrarladı. Sipariş sözleşmesinin tasarımı sıralama diyagramıyla yapılır ve sıralama diyagramına tasarım kalıpları da katılabilir.

11.4 Etkinlik Çubukları

Diyagramdaki dikey sütunların (etkinlik çubuklarının) büyüklüğü önemli değildir. Bunlar bir sınıfın nereden itibaren işleme girdiğini ve diğer sınıflarla nereden itibaren etkileşime girdiğini gösterir.

> **Ek bilgi (derste anlatılmadı):** Slayttaki Denetçi, Uzman ve Yaratıcı kalıpları büyük olasılıkla Larman'ın GRASP kalıplarının Türkçe adlarıdır: Controller, Information Expert ve Creator. Denetçi, sistem olayını kullanıcı arayüzünden alıp ilgili nesnelere yönlendiren sınıftır. Uzman, gerekli bilgiye sahip olan sınıfa sorumluluk verilmesidir. Yaratıcı, bir nesneyi onu içeren ya da kaydeden sınıfın yaratmasıdır.

12. Tasarım Sınıf Diyagramı

12.1 Kavramsal Diyagramdan Farkı

Hoca, tasarım sınıf diyagramı ile kavramsal sınıf diyagramı arasındaki farkı yeniden hatırlattı. Kavramsal sınıf diyagramında, bir sınıfın hangi fonksiyonları ve hangi işlemleri gerçekleştirdiği ayrıntısına girilmez. Tasarım sınıf diyagramında bu ayrıntıya da girilir.

12.2 Kahve Sitesinin Tasarım Sınıf Diyagramı

Altyazıdan çıkan bilgiler şunlardır:

- KahveSitesi ve KahveKatalog sınıfları vardır. KahveKatalog'da yalnızca tanım alma işlemi yapılır.
- Stoktaki her kahve için tanım, fiyat ve kimlik (id) bulunur.
- Sipariş kalemi için bir genelleştirme vardır: standart kalem ve özel kalem. İkisi de sipariş kalemidir, ama farklı alt özellikleri vardır.
  - İkisinde de ne kadar alındığını gösteren miktar bulunur.
  - Standart kalemde tek bir ürün alınır ve onun fiyatı belirlenir (standart fiyat).
  - Özel kalemde birden fazla ürün birleştirilerek fiyat elde edilir.
- Fatura ve ödeme sınıfları vardır.
- Sipariş sınıfında gerçekleştirilen işlemler: siparişin tamamlanması, yeni sipariş eklenmesi, ödeme yapma, toplamı alma ve fatura basma.

Kısa nottaki slayt okumasına göre sınıflar şöyledir:

Sınıf	Öznitelikler	İşlemler
KahveSitesi	adres: String, isim: String	
KahveKatalog		tanimAl()
Kahve	tanim: String, fiyat: Money, id: int	bul()
Denetçi		kahveEkle()
Sipariş	tamamlandı: boolean	tamamlan(), siparişEkle(), ödemeYap(), toplamAl(), faturaBas()
SiparişKalemi (soyut)	miktar: int	
StandartKalem	stdFiyat: Money	altToplamAl()
ÖzelKalem	(BELİRSİZ)	altToplamAl()
Fatura	id: int, tarih: Date	
Ödeme	miktar: Money, tarih: Date	

İlişkiler:

- KahveSitesi → KahveKatalog (1), KahveKatalog → Kahve (1..*).
- KahveSitesi → Denetçi (1), Denetçi → Sipariş (1..*).
- Sipariş → SiparişKalemi (1..*), SiparişKalemi → Kahve (1).
- Sipariş → Fatura (1), Sipariş → Ödeme (1).
- StandartKalem ve ÖzelKalem, SiparişKalemi'nden türer (genelleştirme).

Hoca sepete kahve ekleme işleminin de böylece tasarlanmış olduğunu söyledi ve soru olup olmadığını sordu.

13. Tasarımdan Kodlamaya

13.1 Tasarımın En Az İki Diyagramı

Hoca, sistem analizi ve tasarımı dersinde olduğu gibi tasarımda en az iki diyagram bulunduğunu hatırlattı:

- **Ardışıl (sıralama) diyagramlar:** Hangi sınıflara ait hangi nesnelerin sırayla işleme gireceğini ve bunun hangi mesajlarla yapılacağını gösterir. Hangi tasarım kalıbının hangi işi desteklediği de burada belirtilir.

- **Tasarım sınıf diyagramları:** Mutlaka olmalıdır.

Bunların dışında tasarımı daha da iyileştirecek, hatta analizden başlayarak çizilmesi gereken diyagramlar etkinlik diyagramları ve durum geçiş diyagramlarıdır.

> **Hocanın vurgusu:** Tasarım sırasında ardışıl diyagramlar ve sınıf diyagramlarının yanına etkinlik diyagramları ve durum geçiş diyagramları da eklenmelidir. "Bu şemayı aklınızda tutun."

13.2 Tasarımın Sonunda ve Kodlamada

Tasarımın sonunda sınıflar ve aralarındaki etkileşimler, işbirlikleri ve bağlantılar belirlenmiş olur. Yalnızca metotların nasıl çalışacağını belirleyen algoritmalar belli değildir. Diyagramlara, programın nasıl hazırlanması gerektiğine dair ipucu veren notlar da eklenebilir.

Kodlama aşamasında:

- Sınıf tanımları sınıf diyagramına bakılarak yapılır.
- Metotlar etkileşim diyagramlarına, yani bu derste sıralama diyagramlarına bakılarak yazılır.
- Sonunda her sınıfta hangi işlemin nasıl gerçekleştiği kontrol edilir.

14. Sıralama Diyagramının Yapısı: Basit Örnek

Hoca önce karmaşık örnekler verdiğini, şimdi daha basit bir diyagram üzerinden yapıyı açıklayacağını söyledi. Basit örnekte dört nesne vardır. Nesneler en üste sırayla yerleştirilir. Uygulamaya hangi nesneler giriyorsa, hangileri önemliyse ya da alan modelinde hangileri varsa onların hangi sınıftan olduğu ve nesne adı belirtilir.

Mesaj sırası şöyledir:

Sıra	Gönderen → Alan	Anlamı
Mesaj 1	Nesne 1 → Nesne 2	Nesne 1, nesne 2'yi etkiler.
Mesaj 2	Nesne 2 → Nesne 3	Nesne 2, nesne 3 ile işbirliği yapar.
Mesaj 3	Nesne 3 → Nesne 2	Nesne 3, mesaj 2 ile istenen işlemi yaptıktan sonra geri dönüp bilgi verir.
Mesaj 4	Nesne 1 → Nesne 4	Nesne 1, doğrudan nesne 4 ile etkileşir.

Buradaki mesajlar, tasarım sınıf diyagramında her sınıfın gerçekleştireceği işlemlerdir. Hocaya göre ardışıl diyagramda en önemli şey, nesnelerin nasıl sıralanacağı ve işleme girme sırasının belirtilmesidir.

Diğer özellikler:

- **Kendi kendine mesaj:** Bir nesne kendine mesaj gönderebilir.
- **Koşullu mesaj:** `if` bloğu içindeki mesajlardır. "Nesne 1, bir koşul uygunsa şu işlemi yaparak nesne 2 ile etkileşime girer" diye okunur.
- **Döngü:** "Şu koşul uygun olduğu sürece şu işlemi döngü içinde yapmaya devam et" biçiminde tanımlanabilir.

Hoca son olarak slayttaki bir işbirliği diyagramını gösterdi. Bu diyagramın nesnelerin hangi sırada girdiğini sıralama diyagramındaki gibi göstermediğini, sıralama diyagramı olsaydı nesneler arasında bir sıralama da vermesi gerekeceğini söyledi.

> **Ders sonrası not:** Hoca işbirliği diyagramının sırayı göstermediğini söyledi. UML'de işbirliği diyagramı (UML 2'de iletişim, communication diyagramı) mesajları numaralandırarak sırayı gösterir. Farkı, sırayı zaman ekseninde değil numaralarla vermesidir. Hocanın kastı büyük olasılıkla, nesnelerin yukarıda zaman sırasına göre dizilmemesidir.

15. Sınıflar Arası İlişkiler

Hoca, nesneye yönelik kavramlar dersinden bilinen sınıflar arası ilişkileri hatırlattı. Sınıflar arasındaki bağlantılar üç türlü olabilir: bağlantı (association), kümeleme (aggregation) ve oluşum (composition). Bunlara genelleştirme eklenir.

15.1 Bağlantı ve Çokluk

Bağlantı (association), iki sınıf arasındaki ilişkidir. Tasarım sınıf diyagramında görüldüğü gibi iki sınıf sayısal olarak farklı biçimlerde bağlanabilir. Bu sayılara çokluk (multiplicity) denir. İşaretler "en az .. en çok" anlamına gelir:

Gösterim	Anlamı
`0..1`	En az sıfır, en çok bir.
`1`	Tam olarak bir.
`0..*`	Hiç bağlantı olmayabilir; varsa çok sayıda olabilir.
`1..*`	En az bir olmalı, çok sayıda da olabilir.

Hocanın örneği: Bir sipariş verildiğinde siparişin içinde en az bir kalem olmalıdır, ama birden fazla kalem de olabilir. Bu `1..*` ile gösterilir.

15.2 Kümeleme ve Oluşum

Kümeleme ve oluşum, sınıflar arasındaki parça-bütün ilişkileridir. Hoca "oluşum" yerine "birleşim" demenin de belki daha uygun olduğunu söyledi. Farkları şudur:

- **Kümeleme (aggregation):** Alt parçalar yalnızca o sınıfa ait üyeler değildir. Kendi başlarına da kullanılabilirler. Alt parçaların kendi başına bir anlamı varsa ilişki kümelemedir.
- **Oluşum (composition):** Parçalar yalnızca o nesneyi meydana getirmek için oluşturulur. Kendi başlarına kullanılamazlar. Nesne yok edildiğinde tüm alt parçaları da onunla birlikte yok olmalıdır.

Hocanın örnekleri:

- **Kitap (oluşum):** Kitap nesnesi yok edildiğinde kitabın üniteleri ve bölümleri de yok olur. Ünite ve bölümün ancak kitap varsa bir anlamı vardır.
- **Yığın ve liste (kümeleme):** Yığın (stack) yapısı bir listeyle oluşturulur. Ancak liste sınıfı yığın sınıfına bağlı olmadan da yaratılabilir, kullanılabilir ve başka nesnelere de hizmet verebilir.

Hoca bu sırada bir an "kümeleme ve aggregation arasındaki fark" dedi, hemen "pardon, aggregation ve composition arasındaki fark" diye düzeltti.

15.3 Genelleştirme

Genelleştirme (generalization), ana sınıfla alt sınıflar arasındaki ilişkidir. UML'de içi boş bir üçgenle gösterilir.

Hocanın örneği müşteriydi:

Sınıf	Öznitelikler ve işlemler
Müşteri (genel sınıf)	ad, adres; müşteriye uygulanacak işlemler
Kurumsal müşteri	ad ve adrese ek olarak kredi derecesi, kredi limiti; kredi hesabı işlemleri
Şahsi müşteri	ad ve adrese ek olarak kredi kartı tipi

Kurumsal ve şahsi müşteri, müşteri sınıfının alt sınıflarıdır. İkisine göre yapılan işlem farklı olacaktır.

15.4 Çok Şekillilik ve Soyut Sınıf

Çok şekillilik (polymorphism), farklı sınıflardan yaratılmış nesnelerin aynı mesaja farklı tepki vermesidir. Alt sınıfın miras aldığı bir işlem genellikle olduğu gibi kullanılır. Hoca burada soyut sınıf (abstract class) kavramının da gündeme geldiğini söyledi.

16. Kahve Sitesi Use Case Örneği

Hoca use case'lerin anlaşılıp anlaşılmadığını sordu. Bir öğrenci bir örnek üzerinden geçmenin iyi olacağını söyleyince kullanım senaryosu kahve sitesi için örneklendi.

16.1 Aktörler ve Senaryolar

Nesneye yönelik analizde use case diyagramları kullanılır. Kullanım senaryosu oluşturulurken hangi aktörlerin, hangi senaryoların ve hangi ek tanımlama ve şartların olduğu diyagramla görsel olarak gösterilir; ayrıca metin olarak da verilebilir.

Önce aktörlerin mi senaryoların mı belirleneceği tamamen analizi yapan kişinin kararıdır. Hangisi daha kolay ya da yakın geliyorsa oradan başlanabilir.

16.2 Bilgi Toplama Sonucu

Sistem analizi sırasında toplanan bilgiler şunlardır:

- Site kahve satışı yapar.
- Müşteri hazır kahveleri satın alabileceği gibi özel kahve siparişi de verebilir. Hoca, bunun daha önce sıralama diyagramında görüldüğünü, yani tersten gidildiğini söyledi.
- Bir alışverişte birden fazla kahve siparişi verilebilir.
- Siparişi tanımlamak için müşteri teslimat ve ödeme formlarını doldurur.
- Ödeme kredi kartı ya da banka havalesiyle yapılabilir.
- Bu arada müşterinin bilgileri doğrulanır.

16.3 Adımlar

1. Anlatımlardan yola çıkarak sistemin genel akışı ortaya konur.
2. Aktörler belirlenir: Kahve sitesinden faydalanan müşteri ve kahveyi sağlayan depo.
3. Senaryolar belirlenir. Senaryo, bir aktöre işlevsel bir değer sunan bir bütündür.

Bir aktör olayla ilgileniyor olabilir, ama doğrudan bir senaryoyla ilişkilendirilemiyorsa diyagramdan çıkarılır.

> **Hocanın vurgusu:** Her aktörün bir senaryosu olmalıdır. Kullanım senaryosunda aktörler senaryolarla birleştirilmek zorundadır.

16.4 Diyagram

Müşterinin yaptıkları şunlardır: kahve seçimi yapar, kendi bilgilerini girer, sipariş verir ve siparişi teslim alır. Hocanın anlattığı akış:

- Sipariş vermek için önce kahve seçilmelidir. Müşteri kahve listesine bakar ve seçimini yapar.
- Seçim yaparken "özel kahve oluşturma" adlı bir genişletme yapabilir. Ama bu şart değildir; müşteri hep hazır kahvelerden seçip buraya hiç uğramadan da devam edebilir. Bu yüzden ilişki «extend»'dir.
- Müşteri kahveyi seçtikten sonra sipariş verir. Sipariş verirken mutlaka kendi bilgi girişini de yapmış olması gerekir. Bu yüzden ilişki «include»'dur.
- Siparişin teslimatı ve fatura basımı yapılır. Sipariş teslimatı depodan da sağlanır.

Metin olarak diyagram:

Aktör	Use case	İlişki
Müşteri	Kahve Seçimi	
Müşteri	Özel Kahve Oluşturma	«extend» → Kahve Seçimi
Müşteri	Bilgi Girişi	
Müşteri	Sipariş Verme	«include» → Bilgi Girişi
Müşteri, Depo	Sipariş Teslimatı	
(Depo)	Fatura Basımı	Sipariş Teslimatı ile ilişkili (BELİRSİZ)

Hoca senaryolar arasındaki bağlantıların «include», «extend» ya da özelleştirme olabileceğini tekrarladı.

> **Ders sonrası not:** Hoca bilgi girişi için "Eskiden yapılmışsa buraya uğraması gerekmiyor, ama bunu daha önce yaptığı için kapsamış oluyor" dedi. «include» edilen senaryo, ana senaryo her çalıştığında çalışır. Bilgi girişi yalnızca ilk siparişte yapılıyorsa bu durum büyük olasılıkla bir ön koşulla ("müşteri bilgileri girilmiş olmalı") ya da «extend» ile daha doğru modellenir.

16.5 Senaryo Belgesi

Senaryo belgesi şunları içerir: senaryo adı, ilgili aktörler, senaryonun gerçekleşmesi için ön koşullar, senaryo tamamlandığında sistemin hangi durumda olacağını belirleyen son koşullar, ana akış ve alternatif akışlar.

Slayttaki belgede birincil aktör müşteri, ilgili ise kahve sitesiydi. Hoca, kahve sitesi bir site, yani bir sistem olduğu için dikdörtgenle de çizilebileceğini, ama slaytta çöp adamla çizildiğini söyledi.

16.6 Kahve Sitesinin Kavramsal Sınıf Diyagramı

Hoca, daha önce kavramsal sınıf diyagramının kasa (satış) örneği üzerinden verildiğini, slaytta aynı kahve sitesi için de bir kavramsal sınıf diyagramı bulunduğunu gösterdi. Burada da sınıf kutularının altında o sınıfa uygulanacak mesajların (işlemlerin) tanımı yoktur.

Örnek Sorular

Bu derste ekrandan kare yoktur. Aşağıdaki sorular hocanın derste sözlü olarak sorduğu sorulardır.

Soru 1 (derste sözlü, 00:13:52): Gereksinim mühendisliğinin bilgi toplama aşamasında hangi yolları kullanabiliriz?

> **Cevap (derste verildi):** Yüz yüze görüşme (interview), anket ve yerinde inceleme. Yerinde incelemede süreçlerin gerçekte nasıl işlediği, örneğin 3-4 gün boyunca işin içinde bulunularak izlenir. Bilgiler tüm paydaşların katılımıyla toplanır.

Soru 2 (derste sözlü, 00:13:10): Gereksinim mühendisliğinin başlangıç aşamasında ne yapılır?

> **Cevap (derste verildi):** "Bu işe neden giriyoruz?" sorusu cevaplanır. Gerekçelendirme (justification) yapılır ve fizibilite sonuçları verilir.

Soru 3 (derste sözlü, 01:42:51 ve 01:45:04): Sınıflar arasındaki bağlantı, kümeleme ve oluşum nedir? Kümeleme ile oluşum arasındaki fark nedir?

> **Cevap (derste verildi):** Bağlantı, iki sınıf arasındaki ilişkidir ve çoklukla (`0..1`, `1`, `0..*`, `1..*`) gösterilir. Kümelemede alt parçalar kendi başlarına da kullanılabilir (liste, yığından bağımsız var olabilir). Oluşumda parçalar yalnızca o nesne için vardır; nesne yok edilince parçalar da yok olur (kitap yok edilince ünite ve bölümleri de yok olur).

Soru 4 (derste sözlü, 01:50:21): Çok şekillilik nedir?

> **Cevap (derste verildi):** Farklı sınıflardan yaratılmış nesnelerin aynı mesaja farklı tepkiler vermesidir.

Soru 5 (derste sözlü, 02:00:33, sınav türü soru): Size bir paragraf verilse, bu paragraftan bu derste görülen diyagramlardan (use case, kavramsal sınıf, sıralama, etkinlik, tasarım sınıf diyagramı) herhangi birini çizebilir misiniz? Hangi diyagram analizde, hangisi tasarımda çizilir?

> **Cevap (derste verildi, kısmen):** Analizde use case diyagramları, kullanım senaryosu belgeleri, kavramsal sınıf diyagramı ve gerekirse ER diyagramı çizilir. Tasarımda sıralama diyagramları, tasarım sınıf diyagramı, etkinlik ve durum geçiş diyagramları çizilir. Sıralama diyagramına analizde başlanabilir; etkinlik diyagramı bazı kaynaklarda analizde sayılır. Belirli bir paragraf derste verilmedi; kahve sitesi örneği bu tür bir sorunun çözümü olarak okunabilir.

Dersin Toparlaması

Bu derste önce gereksinim mühendisliğinin yedi adımı tekrar edildi: başlangıç, bilgi toplama, işleme, pazarlık, tanımlama, doğrulama ve yönetim. SRS belgesi ve izlenebilirlik tablosunun projelerde de isteneceği vurgulandı. Ardından nesneye yönelik çözümleme anlatıldı: alan analizi, müşterinin doğal dildeki isteklerinden kullanım öyküsüne geçiş, kullanım senaryosu belgesi ve use case diyagramı. Use case'ler arasındaki «include» ve «extend» ilişkileri, sınıfların ve etkileşimlerin belirlenmesi ve işlemlerin yazılmadığı kavramsal sınıf diyagramı bu kısmın temelini oluşturdu.

Tasarım tarafında sıralama diyagramı, etkinlik diyagramı ve işlemlerin de yazıldığı tasarım sınıf diyagramı kahve sitesi örneği üzerinden gösterildi. Sınıflar arası ilişkiler (bağlantı, kümeleme, oluşum, genelleştirme) ve çok şekillilik hatırlatıldı. Hoca, bir paragraftan bu diyagramlardan herhangi birinin çizilmesinin istenebileceğini, hangi diyagramın analizde hangisinin tasarımda çizildiğini ve aralarındaki bağlantıyı bilmenin önemli olduğunu söyledi. Önümüzdeki hafta durum (state) diyagramları ve tasarım kalıpları işlenecek.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.